World_Playerオブジェクト

Tag	**Player**	Layer	

コンポーネント

Transform
SpriteRenderer
Animator
CircleCollider2D
Rigidbody2D　重力なし、Z軸回転なし
PlayerInput　専用のInputActionアセット

スクリプト

World_PlayerController.cs

シーン「WorldMap」専用のPlayerとしてユーザーの操作で上下左右を自在に動ける。

列挙型を所有

public enum Direction	値：none,left,right	どちらを向いているか

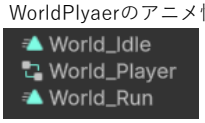
変数

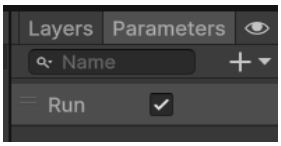
public float speed	値：3.0 f	移動スピード係数
Vector2 moveVec	値：zero	InputSystemからの入力値
float angleZ	値：0.0f	Playerの向き
Rigidbody2D rbody		
Animator animator		
bool isActionButtonPressed;		ボタンが押されていたらtrue
public bool IsActionButtonPressed		外から読み書き可のプロパティ

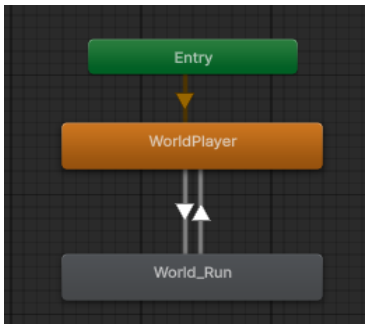
メソッド

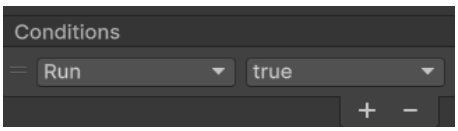
void OnActionButton()	ボタンが押されていたらIsActionButtonPressedプロパティをtrueにする
void OnMove()	Moveアクションの値（Vector2)を変数moveVecに取得
float GetAngle()	InputSystemの入力値moveVecを活用してPlayerの向き変数angleZを導く
Direction AngleToDirection()	自作型Directionを返す。変数angleZをもとにPlayerがどの向きかを示す。
void Start()	変数rbodyと変数animatorを初期化
void Update()	プレイヤーの向きにあわせて絵を左右反転、 キーが押されているか否かでアニメを切り替え(アニメRunプロパティをtrueかfalseか)
void FixedUpdate()	変数moveVec * speedをVelocityに代入

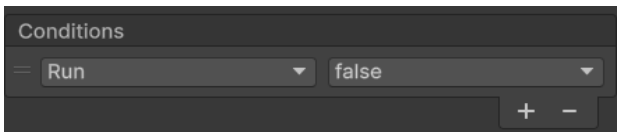
WorldPlyaerのアニメ情報











(WorldMap)MainCameraオブジェクト

Tag	**MainCamera**	Layer	

コンポーネント
Transform
Camera
AudioListener
UmivearsalAdditionalCameraData
PixelPerfectCamera　AssetsPixelsPerUnit 19-20

スクリプト
World_CameraController.cs
シーン「WorldMap」専用のカメラとしてPlayerを追いかける

変数
GameObject player　Playerオブジェクトの参照

メソッド
void Start()　変数playerにPlayerオブジェクト情報を代入
・GameObjectクラスのFindGameobjectWithTag("タグ名")メソッドを利用
void Update()　playerがいるなら、playerのXとY座標を追う（Zは-10固定）

GameManager.cs　に　管理すべきstatic変数の追加

鍵の管理　**public static int keys**

WorldMapでPlayerがEntranceの鍵を開けると減少

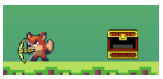
各Stageシーンで鍵の宝箱を開けると増加
※一度入手すると再入手不可

public static Dictionary<"シーン名",bool> keyGot

keyGot

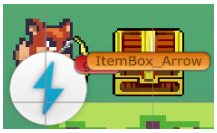
Stage1	TRUE
Stage2	FALSE
…	

各Stageで鍵の宝箱を開けると、対応シーン名のboolをtrue
trueになると宝箱はずっと入手済み状態になる



矢の管理　**public static int arrows**

各Stageシーンで
Playerが矢を打つと1つずつ減少



各Stageシーンで矢の宝箱を開けると増加
※何回でも入手可能

現在どのEntranceを潜ってきたのか、ドア番号の管理
public static int currentDoorNumber

Entranceを潜ってStageに入った時
そのEntranceのドア番号を記憶しておき、
WorldMapに戻ったさいに、対応する
Entranceの場所にいく

(WorldMap)Canvasオブジェクト

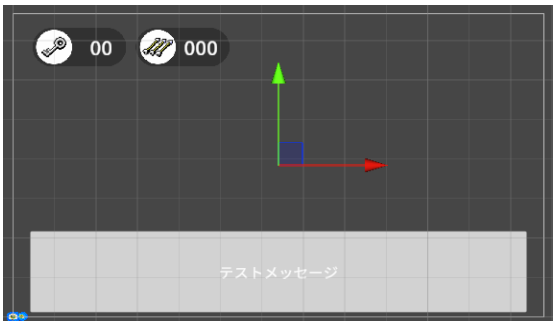
スクリプト
World_UIController.cs
シーン「WorldMap」専用のUI

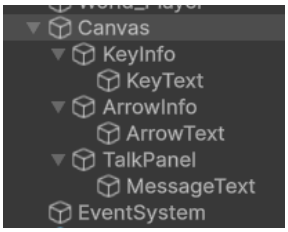
変数

public static Dictionary<int,bool> keyOpened	各エントランスのドア番号が開錠されているかどうか
public TextMeshProUGUI keyText	対象となる鍵のUI
int currentKeys	
public TextMeshProUGUI arrowText	対象となる矢のUI
int currentArrows	
GameObject player	

メソッド

void Start()	すべてのEntranceのオブジェクト情報を取得 もしまだkeyOpenedが初期化されていなければ初期化して、収集した各Entranceのドア番号と開錠状況をkeyOpenedに登録 Playerの位置を決める（GameManagerのcurrentDoorNumberと同じドア番号）
void Update()	把握している所持数(currentKeys、currentArrows)が実際の数（GameManager所持）と異なればUIを更新





Entranceオブジェクト

Tag	**Entrance**	Layer	

コンポーネント

Transform	
SpriteRenderer	
Rigidbody2D	重力なし
BoxCollider2D	isTrigger

スクリプト

EntranceController.cs

プレイヤーと接触した際、アクションボタンが押されていると条件分岐。まず開錠済みかどうかを判断。
未開錠の場合はGamaManagerのKeysを消費して開錠状態にする。Keysが足りなければ不足メッセージが出て終了。
開錠済みの状況でアクションボタンを押すと指定したステージにシーン移行する。
各Entranceは専用の番号doorNumberを保持している。

変数

public int doorNumber		各Entranceに振り分ける番号 1～
public string sceneName		そのEntranceを通して移行したいシーン名
public bool opened	値：false	開錠状態
bool isPlayerTouch	値：false	プレイヤーとの接触状態
bool announcement	値：false	ボタンがおされてアナウンス中状態かどうか
GameObject worldUI		Canvasオブジェクト
Gameobject talkPanel		TalkPanelオブジェクト
TextMeshProUGUI messageText		MessageTextオブジェクト
World__PlayerController worldPlayerCnt		World_PlayerControllerコンポーネント

メソッド

void Start()

変数worldPlayerCnt、worldUI、talkPanel、messageTextの初期化
staticなkeyOpenedディクショナリーの[doorNumber]キーの内容を読み取り
変数openedの開錠(true)/非開錠(false)を決める

void Update()

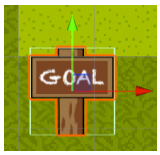
プレイヤーが接触 AND World_PlayerCntrollerにてアクションボタンが押されている
①これからアナウンス状態に入る(announcement == false)
一時ゲーム進行を止める(Time.timeScale = 0)
①－１　開錠済み(opened)
ゲーム進行をもとに戻す
自分のdoorNumberをstaticなcurrentDoorNumberに登録、シーン移行
①－２　(①-1)でなく、鍵がある
"新たなステージへの扉を開けた！"とメッセージを表示、
staticな鍵を減らして、openedを開錠済みにする。
staticなkeyOpenedディクショナリーの[doorNumber]の状態をtrue
announcementをtrue(ボタンを押してアナウンス中という状態にする)
①－3　①-2でもない ※鍵を持っていない
announcementをtrue(ボタンを押してアナウンス中という状態にする)
②すでにアナウンス状態だったら(announcement == true)
ゲーム進行をもとに戻す
メッセージにステージ名を表示
announcementをfalse(アナウンス中状態を解除)
World__PlayerControllerにてアクションボタンのフラグをOFFにしておく

void OnTriggerEnter2D()

Playerタグと接触したら、isPlayerTouchをtrue、パネルを表示（シーン名を表示）

void OnTriggerExit2D()

Playerタグが離れたら、isPlayerTouchをfalse
もしmessageTextがnullでなければ、パネルを非表示、ゲーム進行を再開



接触していないとき：isTouch = false

接触している：isPlayerTouch = true
AND
World_PlayerController側でアクションボタンの入力を検知
（IsActionButtonPressedプロパティを利用）

アナウンス中？

YES → **アナウンスを終了**

NO → 開錠済み

YES → **シーン移行**

NO → 鍵の残数あり

YES → **開錠のアナウンスメッセージ**
鍵を減らす

NO → **鍵不足のアナウンスメッセージ**

単に接触しただけならシーン名が出るだけ
announcement は false のまま

接触時にボタンを押すとアナウンス開始かシーン移行
※シーン移行の場合は後のことは関係なし
announcementをtrueにする
この間、ゲーム進行は止まる

アナウンス中(announcement == true)に
再度ボタンをおすとアナウンスを解除
（announcementをfalseに戻す）
ゲーム進行も再開する

PlayerControllerの変更（体力とダメージの概念）

体力の概念を追加

public static int playerLife　　値：10

static public void PlayerRecovery(int life)
　引数分だけplayerLifeを回復（Lifeの最大値は10）

playerLifeはStart()で10に回復

ダメージ管理

bool playerLife　　ダメージ管理フラグ

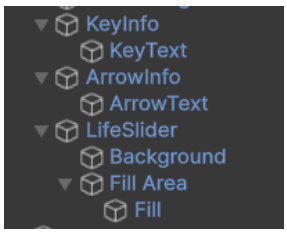
もしダメージ管理フラグが立っていれば
　⇒Update時に点滅表示、何もできない
　⇒FixedUpdate時に何もできない

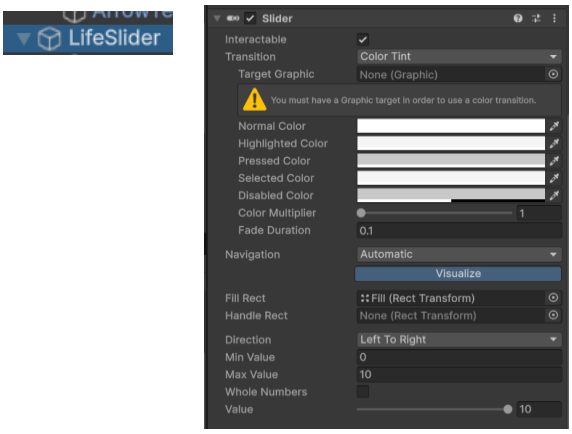
　「Enemy」タグと接触したら**GetDamageメソッド**の発動

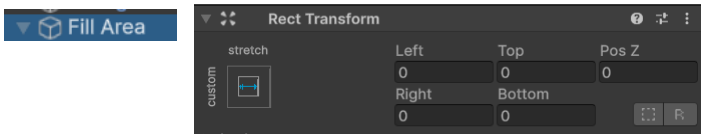
void GetDamage(GameObject target)
　GameState.InGameなら
　　playerLifeを-1
　　もしplayerLifeがまだ残っていれば
　　・ノックバック　・ダメージ管理フラグをtrue
　　・DamageEndメソッドを0.25秒後に発動
　　でなければGameOver

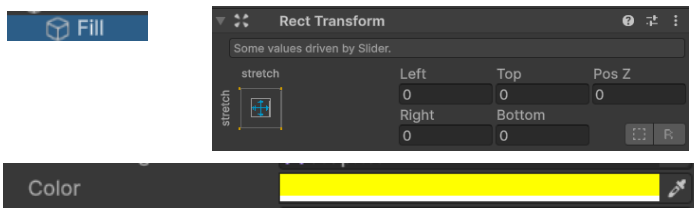
void DamageEndメソッド
　ダメージ管理フラグをfalseしてSpriteRendererを有効化

Canvasプレハブに内容追加









UIController.csの更新

```
public TextMeshProUGUI keyText
int currentKeys
public TextMeshProUGUI arrowText
int currentArrows
public Slider lifeSlider
int currentLife

各currentとGameManagerのstatic、PlayerControllerのstaticを比べて
変化があったらUI更新
```

各アイテムオブジェクト（Arrow、Key、Life）

Tag	**Item**	Layer	

コンポーネント
Transform
SpriteRenderer
CircleCollider2D　isTrigger

スクリプト
Advent_Item.cs
プレイヤーと接触したとき、選んだTypeにあわせた処理を行う
取得後は跳ね上げて消滅する演出

列挙型を所有
public enum AdventItemType　値：None,Arrow,Key,Life　どのアイテムタイプか

変数
public AdventItemType type　値：AdventItemType.None　オブジェクトのアイテムタイプ
public int numberOfArrow　値：10　タイプがArrowだった場合の増加量
public int recoveryValue　値：1　タイプがLifeだった場合の回復量

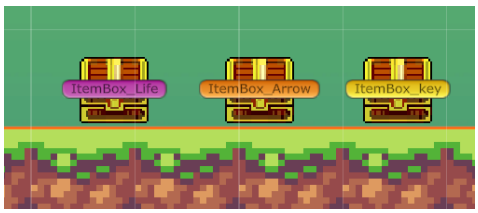
メソッド
void OnTriggerEnter2D()　Playerタグと接触したら、タイプごとに処理を行う（switch文）
・Key⇒鍵を増加、keyGotディクショナリーをtrueに更新
・Arrow⇒矢を変数numberOfArrow分だけ増加
・Life⇒playerLifeを変数recoveryValue分だけ回復
取得演出（Collider無効、重力復活、AddForceで跳ね上げ）

ItemBoxオブジェクト

Tag		Layer	

コンポーネント
Transform
SpriteRenderer
BoxCollider2D　isTrigger

実際にステージに配置されるのはItemBox



スクリプト
Advent_ItemBox.cs
プレイヤーと接触したとき、セットしたアイテムを生成し、オープン状態にする
※鍵以外はステージを読み込むたびに復活する

変数
public Sprite openImage　オープン時の絵
public GameObject itemPrefab　生成するアイテムプレハブ
public bool isClosed　値：true　Closeフラグ
public AdventItemType type　値：AdventItemType.None　収納しているアイテムのタイプ

メソッド
void Start()　タイプがKeyだった場合のみ
GameManagerのkeyGotディクショナリーを調べて、取得済みなら最初からオープン

void OnTriggerEnter2D()　Playerタグと接触したら、Closeフラグをfalseにして絵も変更
アイテムプレハブを生成（Instantiate）